



ENCODE BUILD ON CANTON · CHECKPOINT 2

Veil

Private repo-style financing against tokenized collateral on Canton.

Private Credit

Tokenized Collateral

Selective Disclosure

PROBLEM

Institutional financing needs privacy and shared state

Private credit, repo, OTC secured lending, and treasury financing involve sensitive business data.

1

Counterparties

Borrower and lender identity cannot leak to the market.

2

Terms

Pricing, maturity, collateral, and health are commercially sensitive.

3

Regulators

Auditors need visibility without making deals public.

4

Reconciliation

Offchain emails/PDFs/spreadsheets create operational drag.

`public chain` → everyone sees terms
`offchain ops` → private but fragmented

`needed` → shared workflow state
`needed` → need-to-know visibility
`needed` → auditable authorization

SOLUTION

Veil: private bilateral financing execution

A known lender privately offers financing to a known borrower. The borrower pledges tokenized collateral, accepts the offer, receives principal, repays, and gets collateral released.

Veil is not a discovery marketplace. It is the private execution layer after counterparties agree to transact.

LoanOffer lender signatory
borrower + regulator observe

Accept borrower controls
locks collateral + delivers principal

Repay borrower controls
closes loan + releases collateral

The private financing lifecycle

01 • FUND

Initial holdings

Lender starts with cash. Borrower starts with tokenized T-Bill/MMF collateral. Each holding is visible only to its owner.

02 • OFFER

Private terms

Lender creates a borrower-specific offer. Borrower and regulator observe; outsiders cannot query it.

03 • ACCEPT

Atomic drawdown

Borrower accepts. Collateral locks and principal is delivered in the same Canton transaction.

04 • OBSERVE

Selective disclosure

Regulator gets read-only visibility as an observer without making the deal public.

05 • REPAY

Close facility

Borrower repays principal plus interest. Loan closes and collateral is released back to the borrower.

06 • RISK

Optional liquidation

If collateral value drops below threshold, lender can liquidate under on-ledger LTV rules.

```
Privacy proof: same active-contracts query  
lender/borrower/regulator -> deal visible  
outsider -> []
```

```
Business proof: 100 principal + 5 interest  
150 collateral units locked  
105 repayment releases collateral
```

Concrete repo-style demo terms

100

USDC-equivalent
principal

5

interest

105

repayment amount

150

tokenized T-Bill/MMF
collateral units

Field	Value
Initial LTV	66.7%
Maturity	2026-07-13
Collateral state	Locked while active
Optional risk branch	Price drop -> LTV breach -> liquidation

Privacy is product logic, not infrastructure for its own sake

P

Need-to-know privacy

Lender, borrower, regulator see the deal. Outsider sees nothing.

A

Authorization

Signatories/controllers define who can create, accept, repay, or liquidate.

L

Lifecycle

Offer acceptance locks collateral and delivers principal atomically.

R

Regulator view

Selective observer rights provide audit without public disclosure.

VISIBLE

Lender



VISIBLE

Borrower



VISIBLE

Regulator



EMPTY

Outsider: []

A 3-minute judge path

1

Lender creates a borrower-specific offer with canonical financing terms.

2

Borrower sees the offer. Outsider sees nothing.

3

Borrower accepts; collateral locks and principal is delivered on-ledger.

4

Regulator observes read-only while outsider still gets an empty active-contracts response.

5

Borrower repays 105; loan closes and collateral releases.

6

Optional: price drop creates LTV breach; lender liquidates.

WHAT WORKS TODAY

Built and verified

Daml architecture

`CashHolding` ,
`CollateralHolding` ,
`LoanOffer` , `Loan` , and
`LoanClosed` .

Role-based UI

React/Vite UI over Canton
JSON Ledger API v2 with raw
ledger proof panel.

On-ledger flow

Asset movement for
acceptance/repayment and
guarded liquidation using
lender-submitted mark/LTV.

```
$ dpm build
✓ veil-0.1.0.dar

$ (cd test && dpm build && dpm test)
✓ lifecycle + privacy tests

$ npm --prefix frontend run build
✓ production frontend build
```


From hackathon proof to institutional product

1

Token standard integration

Replace demo assets with Canton token standard / Canton Coin integrations.

2

Marketplace extension

Add `LoanProgram` and `BorrowRequest` for discovery before private execution.

3

Production signing

Add external signing, wallet flows, and production identity/auth.

4

Professional workflows

Add oracle-signed marks, PQS reporting, richer credit terms, margin calls, and compliance flows.

Hackathon scope: prove private institutional financing coordination — not production token infrastructure.